Classical Appearances of π_g: A Geometric Substitution Note

Wayne Baker

July 5, 2026

Abstract

This short note records representative classical formulae in which the usual constant π is rewritten as a geometric limiting constant π_g. The point is not to claim new classical identities. Rather, once a prior geometric argument establishes

$$\pi_g = \pi,$$

every exact occurrence of π may be equivalently written as π_g. The distinction between the completed limiting value π_g and finite approximants $\pi_g^{(N)}$ is emphasized. A numerical consistency test is included using an N-series polygonal approximation to π_g and the Gaussian integral.

1 Purpose of the note

Suppose a geometric construction or polygonal acceleration scheme defines a sequence of finite approximants

$$\pi_g^{(N)}$$

and proves that these approximants converge to the ordinary circle constant:

$$\pi_g = \lim_{N\to\infty} \pi_g^{(N)} = \pi.$$

Then π_g is not a new numerical value different from π. It is an exact geometric representation of the same completed constant. Consequently, any exact classical identity involving π may be rewritten with π_g.

The main distinction is therefore:

$$\pi_g^{(N)} \text{ is finite and approximate,}$$

while

$$\pi_g \text{ is exact, with } \pi_g = \pi.$$

Thus exact substitutions use π_g, whereas numerical tests necessarily use finite approximants $\pi_g^{(N)}$.

2 The substitution principle

Proposition 1 (Geometric substitution principle)**.** *Assume that a geometric limiting constant π_g has been established by*

$$\pi_g = \pi.$$

Then every exact identity of the form

$$F(\pi) = A$$

may be rewritten equivalently as

$$F(\pi_g) = A,$$

provided the expression F is evaluated on the same branch and domain as in the original identity.

Proof. Since $\pi_g = \pi$, substitution gives

$$F(\pi_g) = F(\pi).$$

Therefore, if $F(\pi) = A$, then $F(\pi_g) = A$. This is equality by replacement of identical constants, not a new analytic theorem. □

Remark 1. *The finite approximation $\pi_g^{(N)}$ should not be used in exact identities unless an error term is included. For finite N,*

$$\pi_g^{(N)} \approx \pi_g = \pi,$$

but generally

$$\pi_g^{(N)} \neq \pi.$$

Therefore

$$F(\pi_g^{(N)}) \approx F(\pi),$$

whereas

$$F(\pi_g) = F(\pi)$$

is exact.

3 Representative classical formulae

The following table records standard appearances of π and their equivalent π_g-forms. These entries illustrate substitution sites, not independent numerical verifications; several (e.g. the Gamma value) follow directly from the Gaussian integral entry via standard identities rather than constituting separate checks. The single worked numerical example is given in Section 5.

Identity	**Classical form**	**π_g-form**
Euler identity	$e^{i\pi} + 1 = 0$	$e^{i\pi_g} + 1 = 0$
Gaussian integral	$\displaystyle\int_{-\infty}^{\infty} e^{-x^2}\,dx = \sqrt{\pi}$	$\displaystyle\int_{-\infty}^{\infty} e^{-x^2}\,dx = \sqrt{\pi_g}$
Basel problem	$\displaystyle\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$	$\displaystyle\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi_g^2}{6}$

Identity	**Classical form**	π_g**-form**
Gamma value	$\Gamma\left(\frac{1}{2}\right) = \sqrt{\pi}$	$\Gamma\left(\frac{1}{2}\right) = \sqrt{\pi_g}$
Euler reflection formula	$\Gamma(z)\Gamma(1-z) = \frac{\pi}{\sin(\pi z)}$	$\Gamma(z)\Gamma(1-z) = \frac{\pi_g}{\sin(\pi_g z)}$
Wallis product	$\frac{\pi}{2} = \prod_{n=1}^{\infty} \frac{(2n)(2n)}{(2n-1)(2n+1)}$	$\frac{\pi_g}{2} = \prod_{n=1}^{\infty} \frac{(2n)(2n)}{(2n-1)(2n+1)}$
Leibniz series	$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots$	$\frac{\pi_g}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots$
Fourier-Gaussian identity	$\int_{-\infty}^{\infty} e^{-\pi x^2} e^{-2\pi i x\xi}\, dx = e^{-\pi\xi^2}$	$\int_{-\infty}^{\infty} e^{-\pi_g x^2} e^{-2\pi_g i x\xi}\, dx = e^{-\pi_g \xi^2}$
Stirling formula	$n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$	$n! \sim \sqrt{2\pi_g n}\left(\frac{n}{e}\right)^n$
Cauchy integral formula	$f(a) = \frac{1}{2\pi i}\oint_\gamma \frac{f(z)}{z-a}\, dz$	$f(a) = \frac{1}{2\pi_g i}\oint_\gamma \frac{f(z)}{z-a}\, dz$

Each formula in the right column is exactly equivalent to its classical counterpart once

$$\pi_g = \pi$$

has been established.

4 Finite approximants and why small numerical differences appear

The exact identity is

$$\pi_g = \pi.$$

However, a computer does not evaluate the completed infinite limit directly. Instead it evaluates a finite approximation

$$\pi_g^{(N)}.$$

A typical polygonal quantity is

$$Q_N = \frac{N c_N}{2},$$

where c_N is the chord length of a regular N-gon in the unit circle. The first N-series correction is

$$\widehat{Q}_N = \frac{4Q_{2N} - Q_N}{3}.$$

Higher correction levels may be obtained by repeated even-power cancellation. For example, if

$$R_N^{(0)} = Q_N,$$

then define

$$R_N^{(m)} = \frac{4^m R_{2N}^{(m-1)} - R_N^{(m-1)}}{4^m - 1}.$$

A finite-stage geometric approximation to π_g is then

$$\pi_g^{(N)} = R_N^{(m)}$$

for some finite N and correction order m.

Thus the exact equality is

$$\pi_g = \lim_{N\to\infty} \pi_g^{(N)} = \pi,$$

while the numerical computation only observes

$$\pi_g^{(N)} \approx \pi.$$

Small differences in numerical tables arise from three sources:

$$\text{finite } N,$$

$$\text{finite floating-point precision,}$$

and, for integral tests,

$$\text{finite integration windows.}$$

For example, a numerical Gaussian integral may use

$$\int_{-6}^{6} e^{-x^2}\,dx$$

instead of the exact improper integral

$$\int_{-\infty}^{\infty} e^{-x^2}\,dx.$$

The missing Gaussian tail is very small, but it is not literally zero. Concretely,

$$2\int_{6}^{\infty} e^{-x^2}\,dx = \sqrt{\pi}\,\operatorname{erfc}(6) \approx 3.81 \times 10^{-17},$$

which is roughly two orders of magnitude smaller than the 3×10^{-15} discrepancy observed in the normalized-area test below. The truncation error is therefore not the dominant contributor to that discrepancy; ordinary double-precision floating-point roundoff (accumulated over the chord-doubling recursion, the N-series acceleration, and the Simpson quadrature) accounts for essentially all of it.

5 Gaussian normalization test

The classical Gaussian integral is

$$\int_{-\infty}^{\infty} e^{-x^2}\,dx = \sqrt{\pi}.$$

Using

$$\pi_g = \pi,$$

this becomes

$$\int_{-\infty}^{\infty} e^{-x^2}\,dx = \sqrt{\pi_g}.$$

Equivalently, the normalized geometric Gaussian is

$$G_g(x) = \frac{e^{-x^2}}{\sqrt{\pi_g}},$$

and its exact area is

$$\int_{-\infty}^{\infty} G_g(x)\,dx = 1.$$

A finite numerical test replaces π_g by $\pi_g^{(N)}$ and replaces $(-\infty, \infty)$ by a large finite interval:

$$\frac{1}{\sqrt{\pi_g^{(N)}}} \int_{-6}^{6} e^{-x^2}\,dx \approx 1.$$

Using a second-order N-series value computed from

$$N = 24576, \quad 49152, \quad 98304,$$

one obtains the following sample values.

Quantity	**Computed value**	**Reference/comparison**
Q_{24576}	3.14159264503368973	–
Q_{49152}	3.14159265145076638	–
Q_{98304}	3.14159265305503599	–
$\pi_g^{(N)}$	3.14159265358979267	$\pi \approx 3.14159265358979312$
$\sqrt{\pi_g^{(N)}}$	1.77245385090551588	$\sqrt{\pi} \approx 1.77245385090551588$
$\int_{-6}^{6} e^{-x^2}\,dx$	1.77245385090551033	$\sqrt{\pi_g^{(N)}}$
Difference	$-5.551115123125783 \times 10^{-15}$	0 in the exact limit
Normalized area	0.9999999999999689	1 in the exact limit

The computed normalized area is

$$0.99999999999999689,$$

which differs from 1 by approximately

$$3.11 \times 10^{-15}.$$

This is consistent with ordinary double-precision numerical error, as noted above; the finite integration window contributes negligibly by comparison.

6 Interpretation

The numerical experiment should be interpreted carefully.

It does not prove a new form of the Gaussian integral. The classical theorem already states

$$\int_{-\infty}^{\infty} e^{-x^2}\,dx = \sqrt{\pi}.$$

Because

$$\pi_g = \pi,$$

the corresponding π_g-form is the same theorem written through a geometric limiting constant:

$$\int_{-\infty}^{\infty} e^{-x^2}\,dx = \sqrt{\pi_g}.$$

The numerical calculation simply verifies that a high-accuracy finite geometric approximation $\pi_g^{(N)}$ behaves as expected when substituted into a classical formula involving π.

Thus the purpose of the note is representational:

exact appearances of π may be rewritten as exact appearances of π_g.

The mathematics is unchanged, but the representation emphasizes that the same constant may be obtained as a completed geometric limit.

7 Conclusion

Once

$$\pi_g = \pi$$

has been established, the substitutions in this note are exact. The finite values $\pi_g^{(N)}$ used in numerical tests are approximations to the completed constant, not replacements for it in exact identities.

Therefore the proper hierarchy is

$$\pi_g^{(N)} \approx \pi_g = \pi.$$

The formulae listed above show that the geometric constant π_g, once identified with π, travels consistently through classical analysis, number theory, special functions, asymptotics, Fourier analysis, and complex analysis.

A Python verification script

The following script computes $\pi_g^{(N)}$ from the polygonal/N-series expression and uses it in a Gaussian normalization test.

```
import math


def next_chord_by_doubling(c):
    """
␣␣␣␣Given␣c_N␣=␣chord␣length␣of␣a␣regular␣N-gon␣in␣the␣unit␣circle,
␣␣␣␣compute␣c_{2N}␣using␣only␣square␣roots.

␣␣␣␣Stable␣form:
␣␣␣␣␣␣␣␣c_{2N}␣=␣c_N␣/␣sqrt(2␣+␣2␣sqrt(1␣-␣c_N^2/4))
␣␣␣␣"""
    return c / math.sqrt(2.0 + 2.0 * math.sqrt(1.0 - 0.25 * c * c))


def geometric_Q_values(start_doublings, extra_values):
```

```
    """
␣␣␣␣Start␣from␣the␣regular␣hexagon:
␣␣␣␣␣␣␣␣N␣=␣6,␣c_6␣=␣1.

␣␣␣␣After␣start_doublings,␣return␣Q_N,␣Q_{2N},␣Q_{4N},␣...
␣␣␣␣where
␣␣␣␣␣␣␣␣Q_N␣=␣N␣c_N␣/␣2.
␣␣␣␣"""
    N = 6
    c = 1.0

    for _ in range(start_doublings):
        c = next_chord_by_doubling(c)
        N *= 2

    values = []

    for _ in range(extra_values):
        Q = 0.5 * N * c
        values.append((N, Q))

        c = next_chord_by_doubling(c)
        N *= 2

    return values


def n_series_acceleration(Q_values, order):
    """
␣␣␣␣Even-power␣Richardson/N-series␣acceleration.

␣␣␣␣order␣=␣0:
␣␣␣␣␣␣␣␣Q_N

␣␣␣␣order␣=␣1:
␣␣␣␣␣␣␣␣(4␣Q_{2N}␣-␣Q_N)␣/␣3

␣␣␣␣order␣=␣2:
␣␣␣␣␣␣␣␣applies␣the␣next␣correction␣level␣as␣well.
␣␣␣␣"""
    current = list(Q_values)

    for r in range(1, order + 1):
        factor = 4.0 ** r
        current = [
            (factor * current[i + 1] - current[i]) / (factor - 1.0)
            for i in range(len(current) - 1)
        ]

    return current[0]


def compute_pi_g(start_doublings=12, order=2):
    """
```

```
    Compute pi_g from the geometric polygon/N-series expression.

    start_doublings=12 gives N = 6 * 2^12 = 24576.
    order=2 uses Q_N, Q_{2N}, Q_{4N}.
    """
    values = geometric_Q_values(
        start_doublings=start_doublings,
        extra_values=order + 1
    )

    Q_values = [Q for _, Q in values]
    pi_g = n_series_acceleration(Q_values, order=order)

    return pi_g, values


def simpson(f, a, b, n):
    """
    64-bit Simpson integration.
    """
    if n % 2:
        n += 1

    h = (b - a) / n
    s = f(a) + f(b)

    for i in range(1, n):
        x = a + i * h
        s += (4.0 if i % 2 else 2.0) * f(x)

    return s * h / 3.0


pi_g, source_values = compute_pi_g(start_doublings=12, order=2)

L = 6.0
n_steps = 10_000

integral_approximation = simpson(
    lambda x: math.exp(-x * x),
    -L,
    L,
    n_steps
)

sqrt_pi_g = math.sqrt(pi_g)
difference = integral_approximation - sqrt_pi_g
normalized_area = integral_approximation / sqrt_pi_g

print("Geometric/N-series source values:")
for N, Q in source_values:
    print(f"N = {N:8d}, Q_N = {Q:.17f}")

print()
```

```
print("pi_g␣from␣geometry/N-series:", format(pi_g, ".17f"))
print("sqrt(pi_g):", format(sqrt_pi_g, ".17f"))
print("Integral␣approximation:", format(integral_approximation, ".17f"))
print("difference:", difference)
print("normalized␣area:", format(normalized_area, ".17f"))
```

B Sample console output

```
Geometric/N-series source values:
N =    24576, Q_N = 3.14159264503368973
N =    49152, Q_N = 3.14159265145076638
N =    98304, Q_N = 3.14159265305503599

pi_g from geometry/N-series: 3.14159265358979267
sqrt(pi_g): 1.77245385090551588
Integral approximation: 1.77245385090551033
difference: -5.551115123125783e-15
normalized area: 0.99999999999999689
```

References

[1] L. V. Ahlfors, *Complex Analysis*, McGraw–Hill.

[2] E. T. Whittaker and G. N. Watson, *A Course of Modern Analysis*, Cambridge University Press.

[3] M. Abramowitz and I. A. Stegun, *Handbook of Mathematical Functions*, Dover.

[4] NIST Digital Library of Mathematical Functions, https://dlmf.nist.gov/.